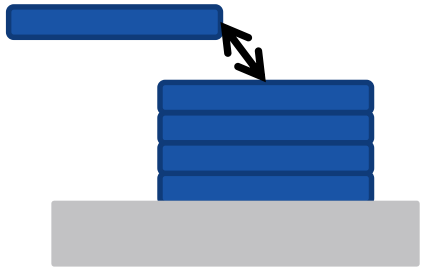
Föreläsning 3

Stack

Föreläsning 3

- ADT Stack
- Stack JCF
- Tillämpning – Utvärdera ett postfix uttryck
- Stack implementerad med en array
- Stack implementerad med en länkad lista

ADT Stack



- Grundprinciper: En stack fungerar som en hög.
- Man fyller på och tar bort överst: **L**ast **I**n **F**irst **O**ut - LIFO.
- Det är en vanlig och viktig datastruktur. Processorn använder tex en för att hålla reda på returadresser vid funktionsanrop.

Måste-metoder:

- pop()
- push(element)

Vanliga metoder:

- empty()
- peak()

Och så behöver vi create()…

Stack JCF

I JCF har vi två alternativ:
Stack som ärver från klassen Vector.
Problem:

- Ärver metoder som en Stack inte ska ha.
- Vector är synkroniserad och bör ej användas om man inte har behov av detta.
- Rekommenderas ej av java api

Deque-interfacet implementeras av LinkedList och ArrayDeque.
Problem:

- Har metoder en stack inte ska ha

Lösning: Skriv en egen. Bäst är då oftast att använda Deque och ArrayDeque.

I kursen när vi behöver en stack för problemlösning är det ok att använda Deque och ArrayDeque trots bristerna.

Stack<E>

java.lang.Object
    java.util.AbstractCollection<E>
        java.util.AbstractList<E>
            java.util.Vector<E>
                java.util.Stack<E>

`boolean empty()`

Tests if this stack is empty.

`E peek()`

Looks at the object at the top of this stack without removing it from the stack.

`E pop()`

Removes the object at the top of this stack and returns that object as the value of this function.

`E push(E item)`

Pushes an item onto the top of this stack.

`int search(Object o)`

Returns the position where an object is on this stack. The top object is in position 1, the next in position 2 ,…

Deque (interface)

Implementeras av LinkedList och ArrayDeque

Har stack-metoderna push, pop, peek och ärver isEmpty.

Det är inte tillåtet att sätta in NULL-element i denna.

Typiskt skapar man denna enligt:

```
Deque<Integer> stack = new ArrayDeque<>();
```

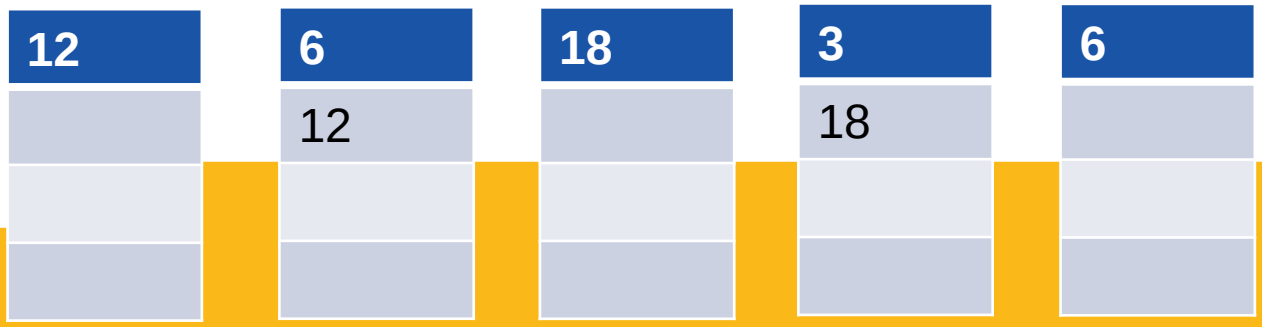
Tillämpning – Omvänd polsk notation

Omvänd polsk notation eller postfix notation var vanlig på miniräknare förr.

- Man kan lättare utföra sammansatta beräkningar utan parenteser
- Man behöver inte ta hänsyn till prioritetsordningen mellan operatorer
- När man ska utföra infix notation (vanlig) gör man det normalt via postfix (även om det kan göras i ett steg mha två stackar, se övning)

Exempel: 12 6 + 3 /

Utvärderas enklast med en stack:



Algoritm för omvänd polsk notation

```
create an empty stack of integers
while there are more tokens get the next token
   if the first character of the token is a digit
      push the token on the stack
   else if the token is an operator
      pop the right operand off the stack
      pop the left operand off the stack
      evaluate the operation
      push the result onto the stack
pop the stack and return the result
```

12 6 + 3 /

	12	6	18	3	6	
		12		18		

För att förstå sig på stackar behöver man framförallt använda dem. Därför ska ni börja med att skriva en metod som utvärderar postfix mha en stack. Här är ett enkelt program som anropar metoden:

```
import java.util.Scanner;

public class TestPostfixEvaluator {

    public static void main(String[] args) {
        PostfixEvaluator evaluator = new PostfixEvaluator();

        String line;
        Scanner in = new Scanner(System.in);
        while(true){
            System.out.println("Enter a postfix expression to evaluate");
            line = in.nextLine();
            if(!line.equals("")){
                try {
                    int result = evaluator.eval(line);
                    System.out.println("Value is " + result);
                } catch (PostfixEvaluator.SyntaxErrorException ex) {
                    System.out.println("Syntax error " + ex.getMessage());
                }
            } else {
                break;
            }
        }
    }
}
```

och här är ett skal för klassen med metoden:

```
import java.util.Stack;
import java.util.EmptyStackException;

public class PostfixEvaluator {

  public static class SyntaxErrorException extends Exception {
      SyntaxErrorException(String message) {
         super(message);
      }
  }
  private static final String OPERATORS = "+-*/";

  private Stack<Integer> operandStack;

  private int evalOp(char op) {
      //hit skickar vi en operator den ska nu utföras på de två översta talen på stacken som ska tas bort. Sedan ska resultatet upp på stacken
  }

   private boolean isOperator(char ch) {
      return OPERATORS.indexOf(ch) != -1;
   }

   public int eval(String expression) throws SyntaxErrorException {
      //skapa en stack för denna beräkning
      String[] tokens = expression.split(" +");//delar upp strängen vid mellanslag
      try {
         for(String nextToken : tokens){
            if (Character.isDigit(nextToken.charAt(0))) {
               // det kommer ett tal så använd Integer.parseInt(nextToken)) och lägg det på stacken!
            }
            else if (isOperator(nextToken.charAt(0))) {
               // det kommer en operator så anropa evalOp för att göra beräkningen med operatorn
            } else {
               throw new SyntaxErrorException("Invalid character encountered");
            }
         }
         //Vi har läst hela uttrycket och gjort alla beräkningar så dags att ta ut svaret som borde vara det enda kvar på stacken
         //Om stacken inte är tom kasta ett syntax error annars returnera resultatet
      } catch (EmptyStackException ex) {
         throw new SyntaxErrorException("Syntax Error: The stack is empty");
      }
   }
}
```

Implementera en stack

Då JCF's stackmöjligheter har brister kommer du oftast vilja implementera en egen. Enklast skapar du då en klass som använder (istället för ärver) en ArrayList eller ArrayDeque som inre datastruktur.

Vi ska här implementera en stack med en array, och en stack med en länkad lista.

Först skapar vi ett interface de båda kan implementera:

```
public interface StackInt<E> {
    E push(E obj);
    E peek();
    E pop();
    boolean empty();
}
```

Stack implementerad med en array

Ska index 0 i arrayen motsvara toppen på stacken?

Nej! Varje gång vi gör pop eller push måste vi då flytta alla element (gäller även om du använder en arrayList).

Istället motsvarar det sista elementet toppen.

- När man lägger till ett element lägger man det bara sist.
- När man tar bort ett element behöver inga andra flyttas.
- Vi behöver bara hålla reda på index för toppen.
- Alla metoder blir O(1).

```java
import java.util.EmptyStackException;
import java.util.Arrays;

public class ArrayStack<E> implements StackInt<E> {
    private E[] data;
    private int top;
    private int maxSize;

    public ArrayStack() {
      top = -1;
      maxSize = 10;
      data = (E[]) new Object[maxSize];
    }
    @Override
    public E push(E obj) {
       if (top == maxSize-1) {
          reallocate();
       }
       top++;
       data[top] = obj;
       return obj;
    }
```

```java
@Override
    public E pop() {
       if (empty()) {
          throw new EmptyStackException();
       }
       return data[top--];
    }
    @Override
    public E peek() {
       if (empty()) {
          throw new EmptyStackException();
       }
       return data[top];
    }
    @Override
    public boolean empty() {
       return top == -1;
    }
    private void reallocate() {
       maxSize*=2;
       data=Arrays.copyOf(data,maxSize);
    }
}
```

Stack implementerad med en länkad lista

- Enkellänkad lista - ändringar endast i toppen (head)
- En stack utnyttjar inte den länkade listans möjlighet att göra ändringar effektivt mitt i
- Stacken har precis rätt storlek
- Referenserna tar upp en plats per data
  - vilket betyder att den kräver dubbla utrymmet jämfört med en full array.

Det normala är att en stack inte bara växer utan också krymper under sin livstid. Därför kommer en arraylist som mest ta fyra gånger mer minne än den behöver (förutsatt att den krymper minnet). En länkad lista kommer istället konstant göra av med dubbelt så mycket minne som en full array.

- Alla operationer blir O(1)

```java
import java.util.EmptyStackException;

public class LinkedStack<E> implements StackInt<E> {

    private static class Node<E> {
        private E data;
        private Node<E> next;

        private Node(E dataItem, Node<E> nodeRef) {
            data = dataItem;
            next = nodeRef;
        }
    }
    private Node<E> top;

    public LinkedStack ()
    {
        top = null;
    }
    @Override
    public E push(E obj) {
        top = new Node<E>(obj, top);
        return obj;
    }
```

```java
    @Override
    public E pop() {
        if (empty()) {
            throw new EmptyStackException();
        } else {
            E result = top.data;
            top = top.next;
            return result;
        }
    }
    @Override
    public E peek() {
        if (empty()) {
            throw new EmptyStackException();
        } else {
            return top.data;
        }
    }

    @Override
    public boolean empty() {
        return top == null;
    }

}
```